CloudEco Experiments and Benchmark Report

Tsung-Yao Chen | 36006750

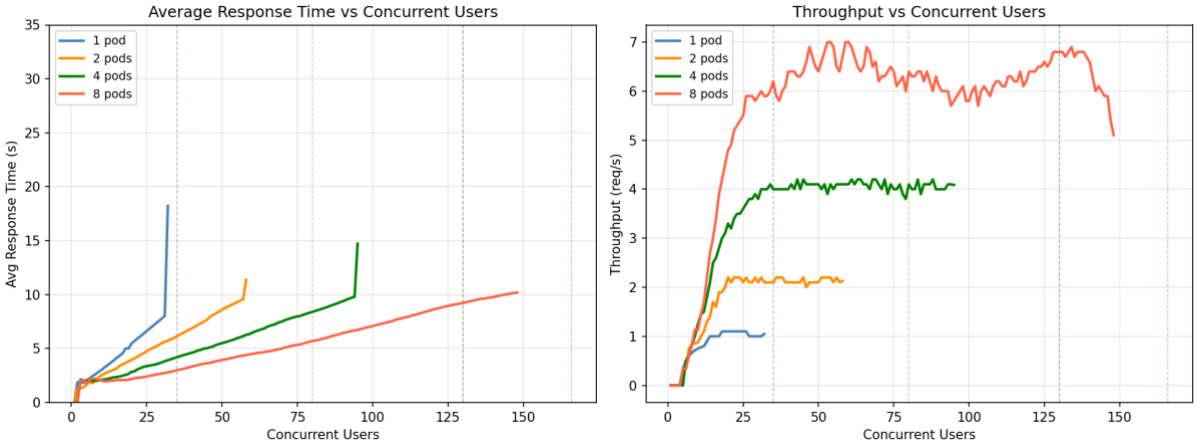
1. Experimental Setup

The CloudEco wildfire detection API (FastAPI + YOLOv8m) was deployed on a 3-node Kubernetes cluster on GCP (australia-southeast1-a): 1 master and 2 worker nodes, each n2-custom-4-8192 (4 vCPU, 8 GB RAM). Each pod was resource-limited to 1.0 vCPU and 2 Gi memory. Load was generated using Locust, mixing /api/predict (weight 2) and /api/annotate (weight 1) requests with a base64-encoded test image and 0.1–0.5 s inter-request wait. Concurrent users were ramped until the breaking point — defined as the first HTTP failure or exponential latency growth.

2. Results Table

Pods	Max Stable Users	Avg Latency at Threshold	Max Throughput (QPS)	Scaling Efficiency
1	~34	8,703 ms	1.10	1.00x
2	~79	13,195 ms	2.20	2.00x
4	~128	10,936 ms	4.10	3.73x
8	~164	9,774 ms	6.90	6.27x

3. Graphical Plots



4. Performance Analysis

Bottleneck identification

The primary bottleneck is CPU-bound YOLOv8m inference. $torch.set\_num\_threads(1)$ confines each inference to a single thread, and with a 1 vCPU limit, the per-pod service rate is $\mu \approx 0.91$ req/s (≈1.1 s per inference).

$ThreadPoolExecutor(max\_workers = 2)$ overlaps I/O phases (request parsing, JSON serialisation) with inference, lifting the effective saturation throughput to ~1.10 QPS per pod before the CPU becomes the hard ceiling.

Little's Law verification

Little's Law: the average number of items in a stable system (L) equals their average arrival rate (λ)multiplied by the average time they spend in the system (W).

At the 1-pod breaking point (λ = 1.10 req/s, W = 8.703 s):

$$L = \lambda \times W = 1.10 \times 8.703 \approx 9.6\ concurrent\ requests\ in\ flight$$

This confirms near-full CPU utilisation ($\rho \to 1$) at saturation, consistent with M/M/1 queuing theory: once arrival rate approaches the service rate, queue length grows unboundedly and latency spikes.

Horizontal scaling characteristics

Throughput scales near-linearly: ×2.00, ×3.73, ×6.27 for 2, 4, and 8 pods respectively. The 8-pod parallel efficiency of 78% (6.27× vs theoretical 8×) reflects CPU contention between co-located pods competing for time slices on shared worker nodes. Breaking points scale proportionally (~35 → ~80 → ~130 → ~166 users), confirming that horizontal pod scaling provides predictable, linear capacity expansion.

Autoscaling justification

According to M/M/c Formulas in queuing theory (Multi-Server): For c servers, $Utilization: \left(\rho = \frac{\lambda}{c\mu}\right)$

Because the service is stateless and CPU-bound, each additional pod adds an independent inference server, directly increasing aggregate service rate (c × μ). This was validated empirically through manual horizontal scaling. For example, at 25 concurrent users, 8 pods maintains ~2.5 s average latency compared to >7 s under a 1-pod configuration, confirming that pod count is the primary lever for capacity expansion.

Generative AI Usage Documentation

All AI interactions were conducted via **Claude (Anthropic)** — used to assist with web service architecture design, Dockerfile optimisation, IaC scripting (Terraform and Ansible), load testing script development, and drafting sections of this report. I reviewed, tested, and modified all generated outputs before use.

1. Web Service Design (main.py)

Prompt:

I'm building a FastAPI web service that runs YOLOv8m inference. It needs two endpoints: POST /api/predict (returns bounding boxes, confidence scores, class labels, and latency metrics as JSON) and POST /api/annotate (returns the annotated image as base64). The request body contains a uuid string and a base64-encoded image. The service must not block the event loop during CPU-bound inference. How should I structure this?

Output:

```python
import base64
import asyncio
from concurrent.futures import ThreadPoolExecutor

import cv2
import numpy as np
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from ultralytics import YOLO

MODEL_PATH = "wildfire-detection/fire-models/fire_m.pt"
model: YOLO = None
executor = ThreadPoolExecutor(max_workers=4)

app = FastAPI(title="Wildfire Detection")

@app.on_event("startup")
async def startup():
    global model
    model = YOLO(MODEL_PATH, task="detect")

class InferenceRequest(BaseModel):
    uuid: str
    image: str

def _run_predict(image_b64: str) -> dict:
    image_bytes = base64.b64decode(image_b64)
    np_arr = np.frombuffer(image_bytes, dtype=np.uint8)
    img = cv2.imdecode(np_arr, cv2.IMREAD_COLOR)
    if img is None:
        raise ValueError("Failed to decode image")

    results = model.predict(img, device="cpu", verbose=False)
    result = results[0]
    boxes = result.boxes
    detections = []
    box_list = []
    for i in range(len(boxes)):
        cls_id = int(boxes.cls[i])
        label = model.names[cls_id]
        x1, y1, x2, y2 = boxes.xyxy[i].tolist()
        prob = float(boxes.conf[i])
        detections.append(label)
        box_list.append({"x": x1, "y": y1, "width": x2-x1, "height": y2-y1, "probability": prob})
    speed = result.speed
    return {
        "count": len(detections),
        "detections": detections,
        "boxes": box_list,
        "speed_preprocess_ms": speed.get("preprocess", 0.0),
        "speed_inference_ms": speed.get("inference", 0.0),
        "speed_postprocess_ms": speed.get("postprocess", 0.0),
    }

def _run_annotate(image_b64: str) -> str:
    image_bytes = base64.b64decode(image_b64)
    np_arr = np.frombuffer(image_bytes, dtype=np.uint8)
    img = cv2.imdecode(np_arr, cv2.IMREAD_COLOR)
    results = model.predict(img, device="cpu", verbose=False)
    annotated = results[0].plot()
    _, buffer = cv2.imencode(".jpg", annotated)
    return base64.b64encode(buffer).decode("utf-8")

@app.post("/api/predict")
async def predict(request: InferenceRequest):
    loop = asyncio.get_running_loop()
    payload = await loop.run_in_executor(executor, _run_predict, request.image)
    return {"uuid": request.uuid, **payload}

@app.post("/api/annotate")
async def annotate(request: InferenceRequest):
    loop = asyncio.get_running_loop()
    annotated_b64 = await loop.run_in_executor(executor, _run_annotate, request.image)
    return {"uuid": request.uuid, "image": annotated_b64}
```

Follow-up Prompt:

My inference latency is inconsistent — sometimes 800ms, sometimes 3s+. I suspect it's because input images vary in resolution. How should I resize images before inference?

Output:

```python
def _decode_image(image_b64: str) -> np.ndarray:
    image_bytes = base64.b64decode(image_b64)
    np_arr = np.frombuffer(image_bytes, dtype=np.uint8)
    img = cv2.imdecode(np_arr, cv2.IMREAD_COLOR)
    if img is None:
        raise ValueError("Failed to decode image")

    max_size = 640
    h, w = img.shape[:2]
    if h > max_size or w > max_size:
        if h > w:
            new_h, new_w = max_size, int(w * max_size / h)
        else:
            new_h, new_w = int(h * max_size / w), max_size
        img = cv2.resize(img, (new_w, new_h))
    return img
```

Modification

- Replaced deprecated @app.on_event("startup") with asynccontextmanager lifespan, and added executor.shutdown(wait=True) on teardown to drain in-flight requests cleanly
- Added torch.set_num_threads(1) to prevent PyTorch spawning extra threads that contend under 1 vCPU — this was identified after benchmarking showed unstable throughput with the default setting
- Reduced max_workers from 4 to 2 after load testing showed 4 workers caused CPU thrashing under the 1 vCPU limit
- Added logging and wrapped endpoints in try/except with proper HTTP error codes
- Added /health endpoint for Kubernetes readiness/liveness probes

2. Dockerfile

Prompt:

Write an optimised Dockerfile for a Python 3.12 FastAPI app that uses PyTorch (CPU-only) and OpenCV. Requirements: multi-stage build to keep the image small, install CPU-only torch separately before other requirements, run as non-root user for security compliance.

Output:

```
FROM python:3.12-slim AS builder

WORKDIR /build
RUN python -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"

COPY requirements.txt .
RUN pip install --no-cache-dir \
        torch==2.5.1 torchvision==0.20.1 \
        --index-url https://download.pytorch.org/whl/cpu && \
    pip install --no-cache-dir -r requirements.txt

FROM python:3.12-slim AS runtime

RUN apt-get update && apt-get install -y --no-install-recommends \
        libgl1 \
        libglib2.0-0 \
    && rm -rf /var/lib/apt/lists/*

RUN useradd --create-home --shell /bin/bash appuser

WORKDIR /app
COPY --from=builder /opt/venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"

COPY main.py .
COPY wildfire-detection/fire-models/fire_m.pt wildfire-detection/fire-models/fire_m.pt

USER appuser
EXPOSE 8000
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Modification

- Added libxcb1 to system dependencies — discovered at runtime that OpenCV requires it on this base image
- Added PYTHONUNBUFFERED=1 and PYTHONDONTWRITEBYTECODE=1 env vars to prevent buffered output and .pyc file generation in the container
- Added --log-level warning to the CMD to suppress per-request access logs under high load, which were adding measurable overhead

3. Infrastructure as Code

Terraform

Prompt:

Write Terraform scripts for GCP to provision a Kubernetes cluster environment: 1 master node and 2 worker nodes using n2-custom-4-8192 machines in australia-southeast1-a, on a custom VPC with subnet 10.0.0.0/24. Use a modular structure with separate network and compute modules.

Output:

```hcl
# main.tf
module "network" {
  source       = "./modules/network"
  network_name = var.network_name
  subnet_name  = var.subnet_name
  subnet_cidr  = var.subnet_cidr
  region       = var.region
}

module "compute" {
  source          = "./modules/compute"
  master_name     = var.master_name
  worker_names    = var.worker_names
  machine_type    = var.machine_type
  zone            = var.zone
  subnet_id       = module.network.subnet_id
}

# modules/network/main.tf
resource "google_compute_network" "vpc" {
  name                    = var.network_name
  auto_create_subnetworks = false
}

resource "google_compute_subnetwork" "subnet" {
  name          = var.subnet_name
  ip_cidr_range = var.subnet_cidr
  region        = var.region
  network       = google_compute_network.vpc.id
}

resource "google_compute_firewall" "allow_internal" {
  name    = "allow-internal"
  network = google_compute_network.vpc.name
  allow {
    protocol = "tcp"
    ports    = ["0-65535"]
  }
  source_ranges = [var.subnet_cidr]
}

# modules/compute/main.tf
resource "google_compute_instance" "master" {
  name         = var.master_name
  machine_type = var.machine_type
  zone         = var.zone
  boot_disk {
    initialize_params { image = "ubuntu-os-cloud/ubuntu-2204-lts" }
  }
  network_interface {
    subnetwork = var.subnet_id
    access_config {}
  }
}

resource "google_compute_instance" "workers" {
  count        = length(var.worker_names)
  name         = var.worker_names[count.index]
  machine_type = var.machine_type
  zone         = var.zone
  boot_disk {
    initialize_params { image = "ubuntu-os-cloud/ubuntu-2204-lts" }
  }
  network_interface {
    subnetwork = var.subnet_id
    access_config {}
  }
}
```

Modification

- Added a separate worker_machine_type variable and separate disk size variables (disk_size_master_gb, disk_size_worker_gb) — master and workers have different storage requirements
- Added a firewall rule to expose NodePort range (30000–32767) for external Locust access, which was missing from the generated output
- Added SSH firewall rule (tcp:22) to allow Ansible to connect to the VMs

Ansible

Prompt:

Write an Ansible playbook to install Kubernetes on Ubuntu VMs. It should: disable swap, install Docker with cri-dockerd as the container runtime, install kubeadm/kubelet/kubectl, initialise the cluster on the master with kubeadm init, and join worker nodes automatically.

Output:

```
- name: Setup all nodes
  hosts: all
  become: true
  tasks:
    - name: Disable swap
      shell: swapoff -a && sed -i '/swap/s/^/#/' /etc/fstab

    - name: Install Docker
      shell: |
        apt-get update -q
        apt-get install -y docker-ce docker-ce-cli containerd.io
      args:
        creates: /usr/bin/docker

    - name: Install cri-dockerd
      shell: |
        wget https://github.com/Mirantis/cri-dockerd/releases/download/v0.3.16/cri-dockerd_0.3.16.3-
0.ubuntu-jammy_amd64.deb
        dpkg -i cri-dockerd_0.3.16.3-0.ubuntu-jammy_amd64.deb

    - name: Install kubeadm, kubelet, kubectl
      shell: |
        curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.29/deb/Release.key | gpg --dearmor -o
/etc/apt/keyrings/kubernetes-apt-keyring.gpg
        echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg]
https://pkgs.k8s.io/core:/stable:/v1.29/deb/ /' > /etc/apt/sources.list.d/kubernetes.list
        apt-get update && apt-get install -y kubeadm kubelet kubectl

- name: Initialise master
  hosts: master
  become: true
  tasks:
    - name: kubeadm init
      shell: kubeadm init --pod-network-cidr=10.244.0.0/16 --cri-socket unix:///run/cri-dockerd.sock
      register: kubeadm_output

    - name: Save join command
      copy:
        content: "{{ kubeadm_output.stdout_lines[-1] }}"
        dest: /tmp/join_command.sh

- name: Join workers
  hosts: workers
  become: true
  tasks:
    - name: Fetch join command
      fetch:
        src: /tmp/join_command.sh
        dest: /tmp/join_command.sh
        flat: yes

    - name: Join cluster
      shell: bash /tmp/join_command.sh --cri-socket unix:///run/cri-dockerd.sock
```

4. Locust Load Testing Script

Prompt:

Write a Locust load testing script for my wildfire detection API. It should load and encode a test image to base64 once at module level, then send POST requests to /api/predict (weight 2) and /api/annotate (weight 1). Each user should wait 0.1–0.5 seconds between tasks. Validate the response structure.

Output:

```python
import base64
import uuid
from locust import HttpUser, task, between

with open("test_image.jpeg", "rb") as f:
    IMAGE_B64 = base64.b64encode(f.read()).decode("utf-8")

class WildfireUser(HttpUser):
    wait_time = between(0.1, 0.5)

    @task(2)
    def predict(self):
        payload = {"uuid": str(uuid.uuid4()), "image": IMAGE_B64}
        response = self.client.post("/api/predict", json=payload)
        if response.status_code != 200:
            response.failure(f"HTTP {response.status_code}")

    @task(1)
    def annotate(self):
        payload = {"uuid": str(uuid.uuid4()), "image": IMAGE_B64}
        response = self.client.post("/api/annotate", json=payload)
        if response.status_code != 200:
            response.failure(f"HTTP {response.status_code}")
```

Modification

- Used os.path.join(os.path.dirname(__file__), "test_image.jpeg") instead of a bare filename so the script works when called from any working directory
- Added timeout=30 per request to match the benchmark's breaking point definition
- Added self.client.headers.update({"Connection": "close"}) in on_start() to prevent connection reuse issues observed under high concurrency

Appendix

Claude is used to assist with web service architecture design, Dockerfile optimisation, IaC scripting (Terraform and Ansible), load testing script development, and drafting sections of this report.

This document lists the primary prompts and representative AI-generated outputs for the main implementation components. It does not constitute a complete transcript of all AI interactions. During development, many additional prompts were used for debugging (e.g., resolving runtime errors, interpreting stack traces), conceptual clarification (e.g., understanding queuing theory, Kubernetes networking), and iterative refinement. These have been omitted as they do not represent substantive code generation.

Additionally, Claude automatically compresses long conversation histories when the context window approaches its limit. A complete verbatim log is therefore not recoverable. The outputs presented here are reconstructed from the final implemented code and my recollection of the key generation steps.

All AI-generated code and text was reviewed, tested, and modified by me prior to submission. The analysis, benchmarking methodology, experimental results, and conclusions in this report are my own work. AI tools were used as productivity aids and not as a substitute for original implementation or academic integrity.